DEVELOPING CODE · DOMAIN 1 - DEVELOPING CODE FOR DATA PROCESSING

Libraries and dependencies that actually resolve

Section 1. Scopes, sources, access modes, and why the install worked yesterday.

SECTION 1 of 10 WEIGHT 22% TOPIC Dependencies

admin@sthithapragna.com

Every library question is really a scope question. **Who can see it, and how long does it last?**

Where a dependency can be attached

SCOPE	WHO SEES IT, AND FOR HOW LONG
Notebook-scoped	One notebook session only. Lost on detach or restart.
Compute-scoped	Every notebook and job on that compute. Reinstalled on restart.
Serverless environment	Tasks that reference the environment key. Declared, not installed.
Init script	Runs on every node at start-up. For things pip cannot do.

ON THE EXAM

Notebook-scoped libraries do not persist across sessions. They must be reinstalled at the start of every session, and whenever the notebook is detached.

The pip magic command

- **%pip install** is the recommended way to install for a single notebook session.
- Only that notebook, and jobs associated with it, can see the library. Neighbours are unaffected.
- Works in notebooks scheduled as jobs. In Scala, SQL or R notebooks, reach it via **%python**.
- On clusters of ten nodes or more it drives extra traffic to the driver. Size the driver accordingly.

> ON THE EXAM
>
> On modern runtimes %pip, %sh pip and !pip all install notebook-scoped. On very old runtimes only %pip behaved consistently.

The install succeeded and the import still fails

- **%pip does not restart the Python process.** The old version may stay loaded in the session.
- Call **dbutils.library.restartPython()** after installing or upgrading, then re-import.
- Upgrading or removing core packages such as IPython can break notebook features outright.
- If the session gets into a strange state, restart the cluster or start a new session to reset it.

> ON THE EXAM
>
> An install that reports success followed by an import of the old version is almost always a missing Python restart.

What you are not allowed to uninstall

- You **cannot** uninstall a library that ships with the runtime, or one installed as a compute library.
- You can revert to that version. If you installed a different one, uninstalling drops you back to the underlying one.
- The **-y** flag is required on an uninstall. Without it the command does not proceed.
- Egg files are not supported from volumes or workspace files. Use wheels, or install from the index.

> ON THE EXAM
>
> Uninstall reverts to the layer beneath. It never removes the runtime's own copy.

Where a package can be installed from

SOURCE	NOTES
Public package index	The default. Pin the version.
Unity Catalog volume	Governed storage for private wheels. The preferred private option.
Workspace files	Supported for wheels held beside your code.
Version control	Install directly from a repository URL.
Private repository	Point at an index URL, with credentials from a secret.
DBFS	Discouraged. Any workspace user can modify what is stored there.

ON THE EXAM

Volumes and workspace files are the recommended homes for private packages. DBFS is called out in the docs as a risk because it is writable by everyone.

Requirements files

- Install a whole set at once with **%pip install -r** and a path to the requirements file.
- Capture the current environment with **%pip freeze** redirected to a file, to reproduce it later.
- The target directory must already exist. Freezing into a new folder fails.
- Keep the file in the workspace or a volume so it is versioned alongside the code that needs it.

ON THE EXAM

A requirements file turns 'it worked on my cluster' into something another person can reproduce exactly.

Why unpinned installs break on serverless

- Databricks explicitly recommends moving away from **unpinned installs**.
- They are **not compatible with the serverless compute architecture**.
- Pin to an exact version. Every install then resolves to the same thing on every run.
- This is also what makes a failure reproducible. An unpinned job fails differently each week.

> ON THE EXAM
>
> If an option installs a package without a version, treat it as suspect. That is now a documented incompatibility, not a style preference.

Credentials without leaking them

- Fetch the token with **dbutils.secrets.get**, then reference it as a variable in the magic command.
- Point at the private repository with an **index URL** option carrying the user and token.
- Add an **extra index URL** for the public index so ordinary dependencies still resolve.
- Never paste a token into a notebook cell. It is then in the source, the history and the repository.

> ON THE EXAM
>
> The same pattern works for private version control repositories. Secret, then variable, never a literal.

Compute library or notebook-scoped

COMPUTE-SCOPED

Everything attached sees it

- Survives the session, reinstalled on restart
- One definition for every job on the cluster
- Changing it affects everyone else too
- Right for a shared production dependency

NOTEBOOK-SCOPED

One session, isolated

- Gone on detach or restart
- Neighbours on the cluster are unaffected
- Safe place to try a different version
- Right for exploration and one-off work

THE TIEBREAKER

A production job that must not depend on someone's session should never rely on a notebook-scoped install.

Environments instead of installs

- On serverless you **declare** dependencies in an environment rather than installing them.
- A job task names an **environment key**; the environment carries a version and a dependency list.
- In a notebook, the **Environment side panel** is the reproducible place to declare dependencies.
- A notebook task still cannot take an environment spec. Use a notebook-scoped install there.

> ON THE EXAM
>
> Declared beats installed. The environment is part of the definition, so it is deployed with the job rather than configured on a cluster.

Base environments and faster installs

- A workspace admin can define a **base environment**: a version plus a standard set of dependencies.
- It can be exported as a file and stored in a workspace folder or a volume, so teams share one baseline.
- On recent serverless environment versions, **%uv pip** is a faster alternative sharing the same virtual environment.
- It does not check conflicts against preinstalled packages, and cannot fully remove them.

> ON THE EXAM
>
> Use the faster tool for interactive iteration. Use the environment panel when you need the result to be reproducible.

What the compute will let you install

ACCESS MODE	WHAT IT MEANS FOR LIBRARIES
Standard	Shared between users. JARs, Maven coordinates and init scripts need an allowlist.
Dedicated	Assigned to one user or group. The assigned principal's identity is used.
Serverless	No cluster to install onto. Dependencies are declared in an environment.

ON THE EXAM

Standard was formerly called shared, and dedicated was formerly single user. Both names appear in older material and in the question bank.

What it does, and what it does not do

- Controls which init scripts, JAR files and Maven coordinates may run on **standard access mode** compute.
- It is **empty by default and cannot be disabled**. Modifying it needs the MANAGE ALLOWLIST privilege.
- Allowlisting a path grants **permission to use it**, not permission to read it. READ VOLUME is still required.
- Prefix matching applies. A trailing slash scopes the entry to everything inside that directory.

> ON THE EXAM
>
> Two separate gates: the allowlist says the artefact may be used, and Unity Catalog permissions say whether you may read it. Both must pass.

The Java-side rules

- On standard access mode with Unity Catalog, JAR classes **must be in a named package**. Default-package classes fail on import.
- Libraries used as **JDBC drivers or custom Spark data sources** require ANY FILE permission.
- Maven coordinates go on the allowlist the same way paths do, and can be allowed by group.
- Some libraries write to one shared temporary directory for all users, which undermines isolation.

ON THE EXAM

An import that fails only on shared compute, and works on a dedicated cluster, points at the named-package rule or the allowlist.

When pip is not enough

- Init scripts run on **every node** as compute starts. For system packages, agents, certificates, drivers.
- **Cluster-scoped is recommended** over global. Global scripts run on every cluster in the workspace.
- Store them in workspace files, a Unity Catalog volume, or cloud object storage.
- On standard access mode they must be allowlisted, like JARs.

> ON THE EXAM
>
> Reach for an init script only when the need is genuinely below the Python layer. A package install is simpler and easier to reason about.

Diagnosing a dependency failure in order

01

Which scope

Was it installed for the notebook, the compute, or declared in an environment?

02

Which session

Notebook-scoped installs are gone after a detach. Was the session restarted?

03

Was Python restarted

An install without a restart leaves the previously imported version in place.

04

Which access mode

Standard mode blocks JARs, Maven and init scripts that are not allowlisted.

05

Which version

Unpinned installs resolve differently over time. Pin, then reproduce.

ON THE EXAM

Work down this list rather than guessing. Almost every library question in this exam is answered by one of these five.

What to carry into the exam

- Scope decides visibility and lifetime. Notebook-scoped installs **do not survive a detach**.
- An install that succeeded but imports the old version means **Python was not restarted**.
- You cannot uninstall a runtime or compute library. You revert to it.
- Standard access mode needs an **allowlist** for JARs, Maven and init scripts. It is not a permission.
- Unpinned installs are incompatible with serverless. Pin the version.

> ON THE EXAM
>
> Production dependencies belong in something deployed with the job. If an answer depends on someone remembering to run a cell, it is wrong.

A scheduled job runs a notebook that begins with a cell installing a specific parsing library. It has run nightly without incident for months.

An engineer adds the same library as a compute library on the shared cluster to make it available to a second team, then removes the install cell from the notebook to tidy it up. The nightly job now fails with an import error.

What is the most likely explanation?

A The job runs on its own job compute, which never received the compute library.

B Compute libraries are only visible to interactive notebooks, not to scheduled jobs.

C Removing the install cell also removed the library from the cluster.

D The compute library needs a Python restart that only a notebook can trigger.

◆ PAUSE AND DECIDE

What is the most likely explanation?

A The job runs on its own job compute, which never received the compute library.

The library was added to the shared interactive cluster. The job's compute is a different machine entirely.

B Compute libraries are only visible to interactive notebooks, not to scheduled jobs.

They are visible to everything attached, jobs included.

C Removing the install cell also removed the library from the cluster.

Editing a notebook does not uninstall anything.

D The compute library needs a Python restart that only a notebook can trigger.

Compute libraries are present from start-up. No restart is involved.

ON THE EXAM

Where a library was installed and where the job actually runs are two different questions. Check both.

A developer runs an install command to upgrade a serialisation library from version 3.1 to 3.4 in their notebook. The output ends with a success message naming 3.4.

In the very next cell they import the library and print its version. It reports 3.1. Re-running the install reports that 3.4 is already satisfied.

What should they do?

A Install the library as a compute library instead, since notebook installs cannot upgrade.

B Call dbutils.library.restartPython(), then import the library again.

C Uninstall version 3.1 first, then install 3.4.

D Detach and reattach the notebook, then re-run the install cell.

◆ PAUSE AND DECIDE

What should they do?

A Install the library as a compute library instead, since notebook installs cannot upgrade.

They can. And this would change the version for everyone else on the cluster.

B Call dbutils.library.restartPython(), then import the library again.

The install did land. The already-imported module stayed loaded because the Python process was never restarted.

C Uninstall version 3.1 first, then install 3.4.

The install already succeeded, as the second run confirms. Removal is not the problem.

D Detach and reattach the notebook, then re-run the install cell.

Works, but throws away the whole session state to solve a one-line problem.

ON THE EXAM

Success on install plus the old version on import is the signature of a missing Python restart.

A team stores a proprietary wheel in a Unity Catalog volume and installs it successfully on a dedicated cluster. They then try the same wheel, and a JDBC driver JAR from the same volume, on a standard access mode cluster shared across two teams.

An administrator has already granted the group READ VOLUME on the volume. The JAR install still fails.

What is required to make the JAR install succeed?

A Nothing more. READ VOLUME is sufficient; the failure must be a transient error.

B Move the JAR to DBFS, which standard access mode clusters can read without extra configuration.

C Add the path to the Unity Catalog allowlist, and grant ANY FILE for use as a JDBC driver.

D Switch the cluster to dedicated access mode, since JARs are unsupported on standard mode.

◆ PAUSE AND DECIDE

What is required to make the JAR install succeed?

A Nothing more. READ VOLUME is sufficient; the failure must be transient.

Read permission and allowlisting are two separate gates. Both must pass.

B Move the JAR to DBFS, which standard access mode clusters can read without extra configuration.

DBFS is discouraged precisely because any workspace user can modify it.

C Add the path to the Unity Catalog allowlist, and grant ANY FILE for use as a JDBC driver.

Standard mode requires allowlisting for JARs, and driver use carries the additional ANY FILE requirement.

D Switch to dedicated access mode, since JARs are unsupported on standard mode.

They are supported. Abandoning shared compute to avoid a governance step is the wrong trade.

ON THE EXAM

Two gates, always: the allowlist decides whether the artefact may be used, and permissions decide whether you may read it.

A nightly job on serverless compute has run reliably for a year. Its notebook installs three libraries without specifying versions. Last night it failed with an error deep inside one of them, and nothing in the team's code had changed.

The team wants the job to stop failing, and wants any future failure to be reproducible on demand.

Which change best addresses both requirements?

A Add a retry policy to the task so transient dependency failures resolve themselves.

B Pin every dependency to an exact version, declared in the job's environment.

C Move the installs into an init script so they run before the notebook starts.

D Install the libraries as compute libraries on a dedicated cluster instead.

◆ PAUSE AND DECIDE

Which change best addresses both requirements?

A Add a retry policy so transient dependency failures resolve themselves.

A new version will fail on every retry. It also hides the cause.

B Pin every dependency to an exact version, declared in the job's environment.

Pinning stops the silent upgrade, and declaring it in the environment makes it part of the deployed job.

C Move the installs into an init script so they run before the notebook starts.

Serverless has no cluster for an init script, and unpinned installs would still drift.

D Install as compute libraries on a dedicated cluster instead.

Abandons serverless to work around a versioning problem, and still nothing is pinned.

ON THE EXAM

Nothing changed in the code, so something changed outside it. Unpinned dependencies are the usual answer to that sentence.

> Two analysts share a standard access mode cluster. One needs version 2.0 of a statistics package for an experiment. The cluster has version 1.8 installed as a compute library, which four production jobs depend on.
>
> The analyst must not disturb those jobs, and needs the newer version only for the current piece of work.

What should the analyst do?

A. Install 2.0 notebook-scoped in their own notebook, accepting it is lost when the session ends.

B. Upgrade the compute library to 2.0 and ask the job owners to test against it.

C. Uninstall the compute library, then install 2.0 notebook-scoped.

D. Add 2.0 to the allowlist so both versions can be resolved at once.

◆ PAUSE AND DECIDE

What should the analyst do?

A Install 2.0 notebook-scoped in their own notebook, accepting it is lost when the session ends.

Isolation is exactly what notebook scope provides. Nothing else on the cluster is affected, and impermanence is fine for an experiment.

B Upgrade the compute library to 2.0 and ask the job owners to test against it.

Changes the version for four production jobs to serve one experiment.

C Uninstall the compute library, then install 2.0 notebook-scoped.

Breaks the four jobs, and a compute library cannot be uninstalled from a notebook anyway.

D Add 2.0 to the allowlist so both versions can be resolved at once.

The allowlist governs JARs, Maven and init scripts. It does not resolve Python versions.

ON THE EXAM

Impermanence is a feature here, not a drawback. An experiment that vanishes with the session is exactly what was wanted.

> A platform team must make an internal wheel available to every production job across three workspaces. The wheel is rebuilt weekly, must not be readable by contractors, and installs must not require anyone to edit a notebook.
>
> They currently email the file around and install it by hand.

Which approach best meets all three requirements?

A. Upload the wheel to DBFS in each workspace and reference it from each job definition.

B. Store it in a Unity Catalog volume with permissions, and reference that path from the job definitions.

C. Have each notebook install it from an internal repository at the top of every run.

D. Add a global init script in each workspace that installs the wheel on every cluster.

◆ PAUSE AND DECIDE

Which approach best meets all three requirements?

A Upload the wheel to DBFS in each workspace and reference it from each job definition.

Fails the access requirement outright. Any workspace user can read and modify DBFS.

B Store it in a Unity Catalog volume with permissions, and reference that path from the job definitions.

Governed storage gives real permissions, and referencing it from the job definition removes the manual step.

C Have each notebook install it from an internal repository at the top of every run.

Puts the dependency back inside a notebook cell, which was explicitly ruled out.

D Add a global init script in each workspace that installs the wheel on every cluster.

A global script runs on every cluster in the workspace. Enormous blast radius for one team's package.

ON THE EXAM

Three requirements: governed access, weekly refresh, no manual step. Test each option against all three rather than the first one.